สัปดาห์ที่ 2 · 05_DAG_Planner

# DAG Planner Skill

## แตกงานเป็นบล็อก · แต่ละบล็อกใช้สกิล · DAG + Gantt · รันจริง

สกิลตัวที่ 4 ต่อจาก **yolo · cyolo · agent-team** ให้ AI วางแผนว่าใครทำบล็อกไหน เมื่อไร แล้วส่งให้ทีมรันจริง



โจทย์เว็บ "เหมียวช็อป" · 8 บล็อก · ทีมรันจริง

**1.7** → **2.8** → **2.8**

นาที · แผนแรก | นาที · รันจริง | นาที · แผนหลังเรียนรู้ k

8/8 บล็อกเสร็จ · ตะกร้าจำได้หลังรีเฟรช · แผนรอบสองตรงกับของจริง

สารบัญ

# บทนี้มี 4 ส่วน

1

## ปัญหา + ทวน

มีทีม แต่ไม่มีแผน · DAG · rank · Gantt

2

## บล็อก = สกิล

6 สกิลต่อบล็อก · หน้าตาของบล็อก

3

## สกิล dag-planner

สมอง · มือ · ทีม · แผน vs จริง · k

4

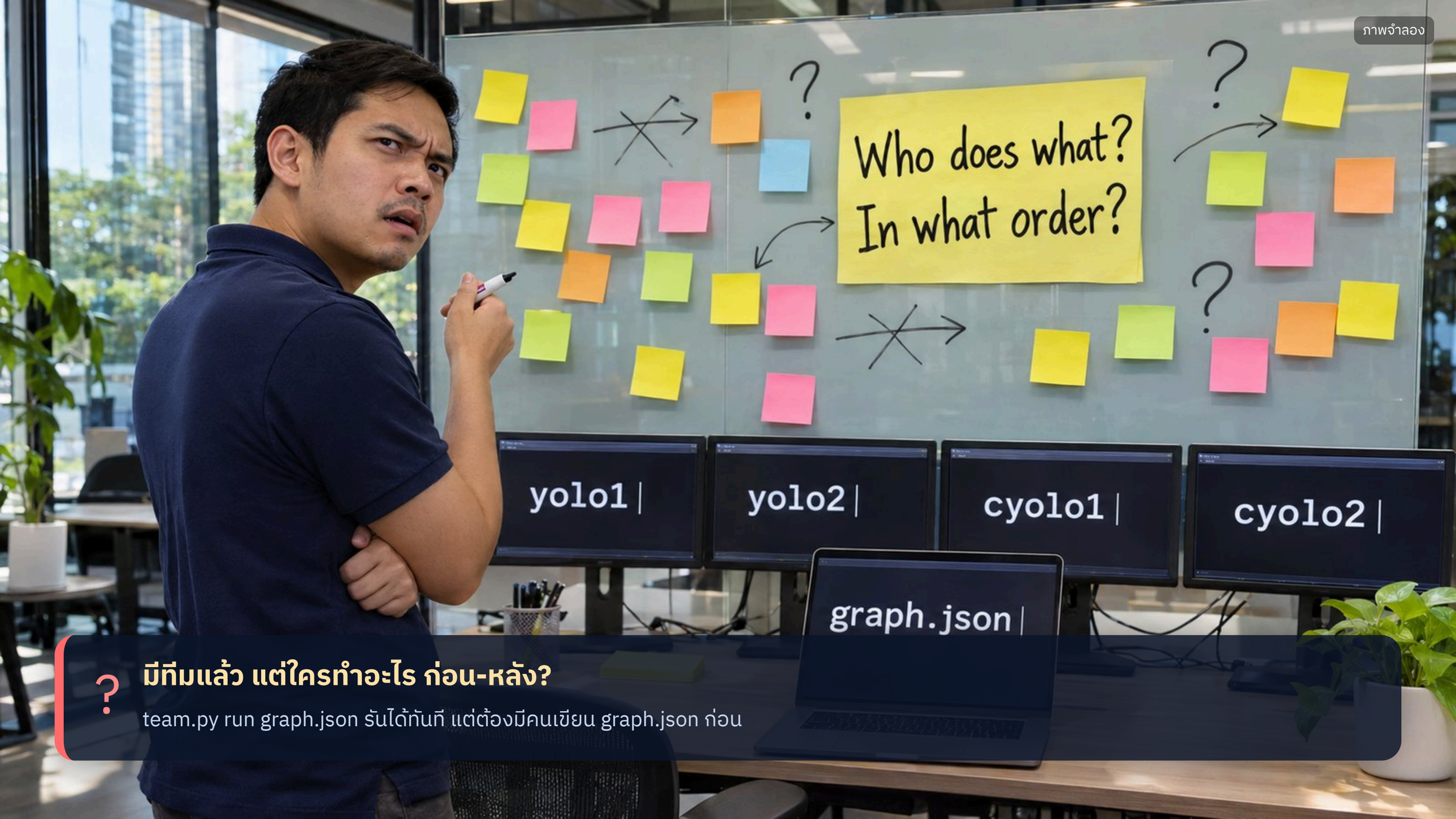
## ลงมือ 5 ขั้น

สร้าง → ติดตั้ง → วางแผน → รันจริง → what-if

ส่วนที่ 1 จาก 4

# ปัญหาและทวนทฤษฎี

บท 04 ให้ **ทีม** · บทนี้ให้ **แผน** ว่าใครทำอะไร ก่อน-หลัง เมื่อไร




**มีทีมแล้ว แต่ใครทำอะไร ก่อน-หลัง?**

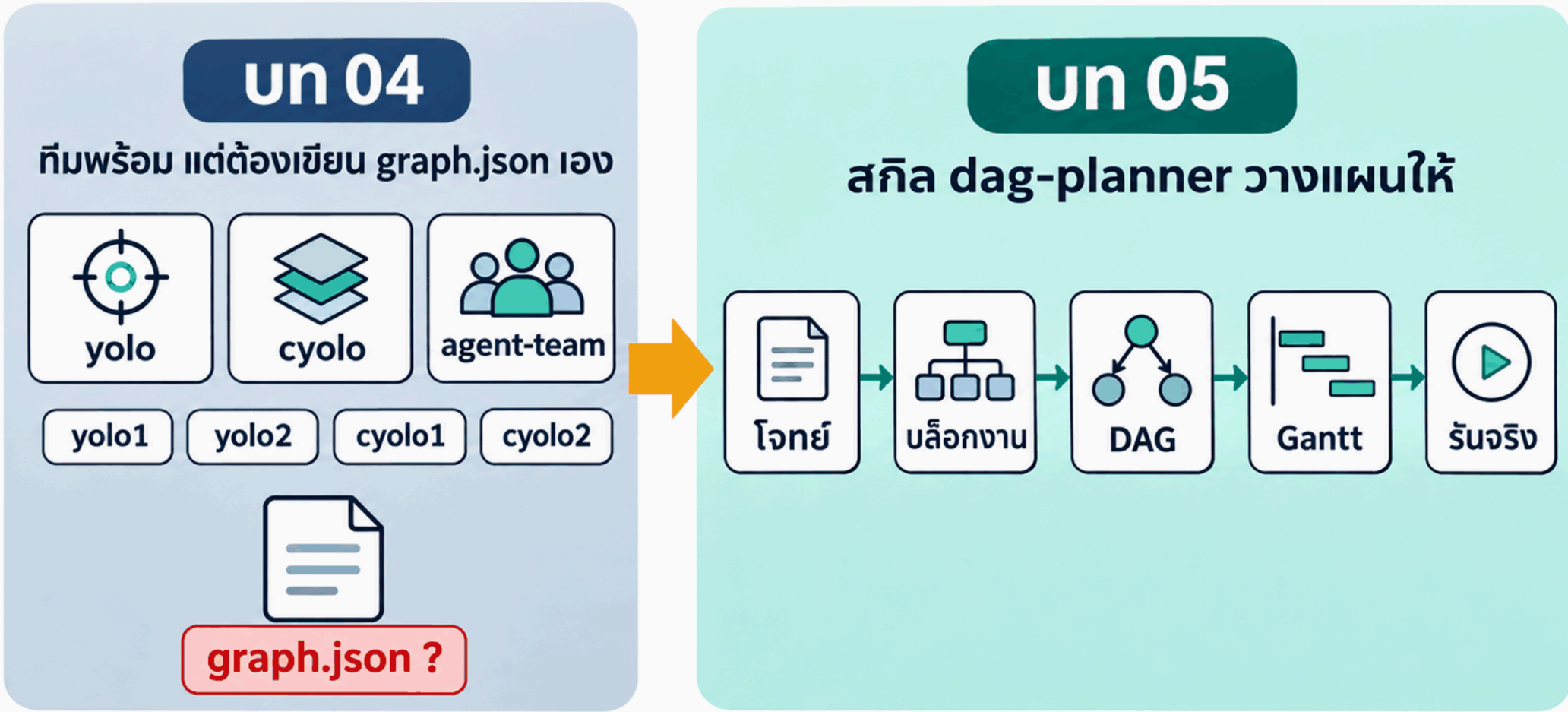
team.py run graph.json รันได้ทันที แต่ต้องมีคนเขียน graph.json ก่อน

1 · ปัญหา

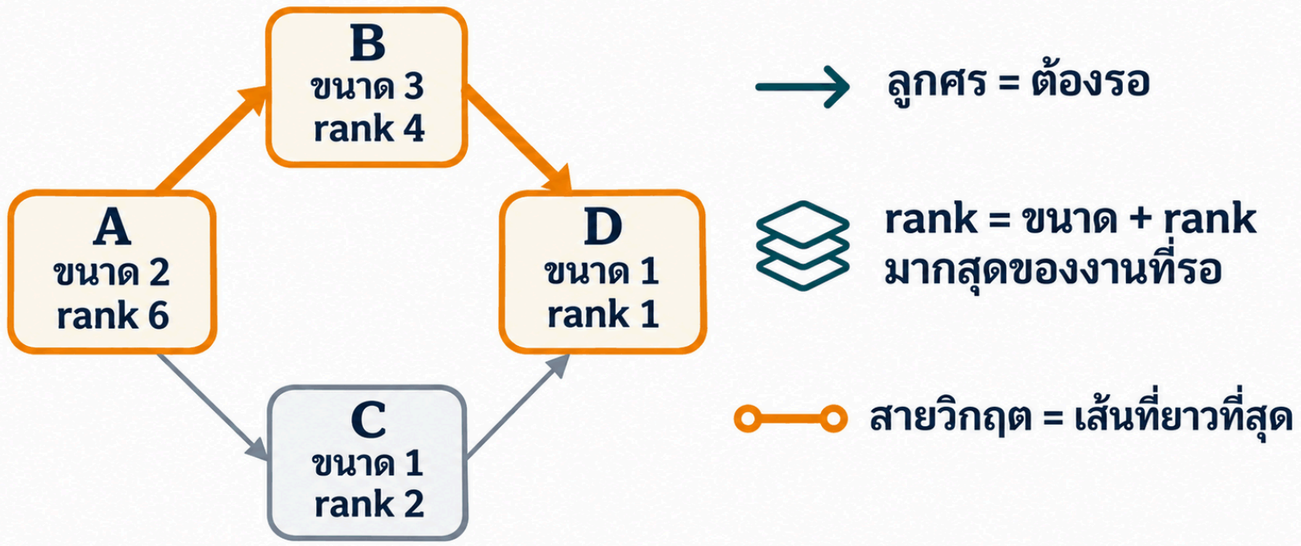
# ก่อนเขียน graph.json ต้องตอบให้ได้

| คำถาม | ถ้าตอบไม่ได้ |
| --- | --- |
| งานมีกี่ชิ้น ชิ้นไหนต้องรอชิ้นไหน | สั่งงานผิดลำดับ |
| แต่ละชิ้นควรใช้สกิลไหน | ใช้บัญชีแพงกับงานง่าย หรือไม่มีใครตรวจงานสำคัญ |
| ชิ้นไหนให้บัญชีไหน เมื่อไร | บัญชีโควตาน้อยรับงานใหญ่จนค้าง |
| ทั้งโปรเจกต์จะเสร็จเมื่อไร | ไม่รู้ว่าเพิ่มบัญชีแล้วเร็วขึ้นไหม |

# บท 04 มีทีม · บท 05 มีแผน



# ทวน: DAG · rank · สายวิกฤต



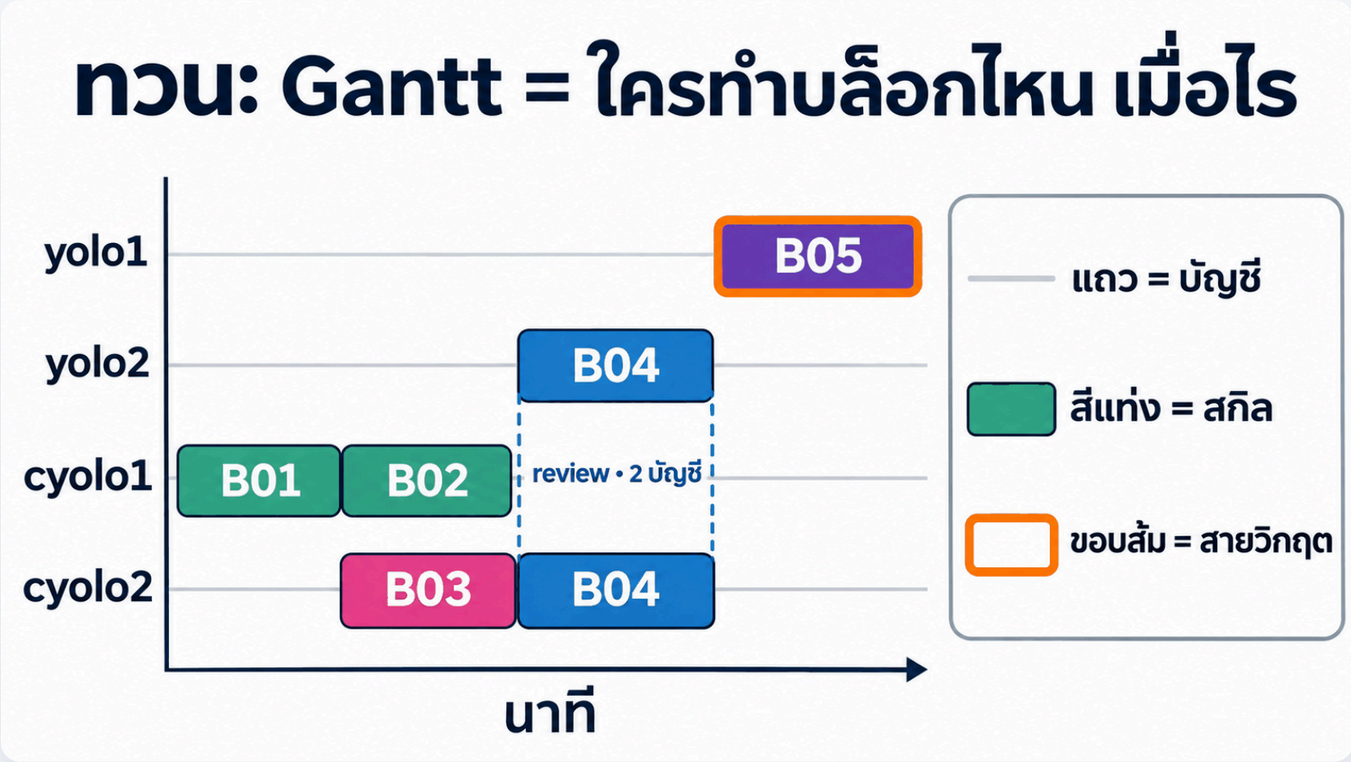

**rank(j) = $c_j$ + max rank(งานที่รอ j)**
คิดย้อนจากงานสุดท้าย

rank(D) = 1 · rank(B) = 3 + 1 = 4

rank(C) = 1 + 1 = 2

rank(A) = 2 + max(4, 2) = **6**

สายวิกฤต = เส้นที่ยาวที่สุด (A → B → D) ได้เลือกบัญชีก่อน




- แถว = บัญชี · แท่ง = บล็อก · **สีแท่ง = สกิล**
- บล็อกหลายบัญชีมีหลายแถว
- เวลารวมสั้นกว่าสายวิกฤตไม่ได้
- ใครทำบล็อกไหน = **KC-QUEST** (บท 06)

ส่วนที่ 2 จาก 4

# บล็อกงาน = การ์ดที่บอกว่าใช้สกิลอะไร

แต่ละบล็อกระบุ **skill** + **difficulty** + ไฟล์ผล + เกณฑ์เสร็จ




**ใบสั่ง = บล็อก · สถานี = สกิล**

บางจานต้องมีคนชิมก่อนเสิร์ฟ (review) · บางจานให้หลายคนเสนอแล้วเลือก (best_of)

# แต่ละบล็อกเลือกสกิลที่ใช้ทำ

## yolo
เขียนโค้ด • งานคิดยาว

Claude 1 บัญชี

## cyolo
เขียนโค้ด • ข้อมูล

Codex 1 บัญชี

## image
สร้างภาพ

Codex 1 บัญชี

## review
ทำ ↔ ตรวจ จนผ่าน

2 บัญชี • คนละ CLI

## debate
Claude vs Codex + ผู้ตัดสิน

3 บัญชี

## best_of
หลายคนทำ เลือกผลดีสุด

N+1 บัญชี

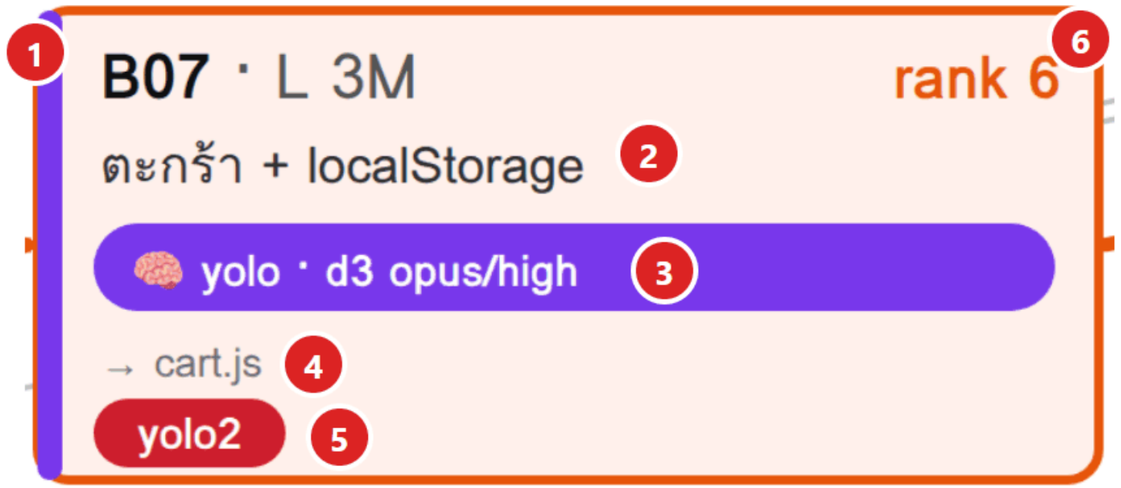
## 2 · สกิลของบล็อก → สิ่งที่ทีมทำจริง

| skill | ใช้เมื่อ | agent-team ทำจริง | บัญชี |
|---|---|---|---|
| 🧠 `yolo` | เขียนโค้ด/เอกสาร งานคิดยาว | งานเดี่ยวฝั่ง Claude | 1 |
| ⚙️ `cyolo` | โค้ด/ข้อมูลตรงไปตรงมา | งานเดี่ยวฝั่ง Codex | 1 |
| 🖼️ `image` | สร้างภาพ | งานภาพ (Codex เสมอ) | 1 |
| 🔍 `review` | งานสำคัญต้องมีคนตรวจ | ผู้ทำ ↔ ผู้ตรวจคนละ CLI | 2 |
| ⚖️ `debate` | ตัดสินใจที่มีถูก/ผิด | Claude vs Codex + ผู้ตัดสิน | 3 |
| 🏆 `best_of` | งานสร้างสรรค์ | หลายคนทำ + ผู้ตัดสิน | N+1 |

review · debate · best_of ใช้หลายบัญชี → ใช้เฉพาะบล็อกที่คุ้ม

# บล็อกงาน 1 บล็อกบอกอะไรบ้าง

บล็อก B07 จากแผน meow-shop (ภาพจริงจาก dag.png)




1. **id · ขนาด**
   B07 · L = ประเมินไว้ 3 ล้านโทเคน (S 0.6 · M 1.5 · L 3)
2. **ชื่องาน**
   ตะกร้า + localStorage
3. **สกิล + ความยาก**
   yolo = Claude 1 บัญชี · ความยาก 3 → opus / high
4. **ไฟล์ผล (output)**
   cart.js ใน app/ บล็อกถัดไปอ่านไฟล์นี้ต่อ
5. **บัญชีที่ KC-QUEST แจก**
   yolo2 (เติมหลังสั่ง plan)
6. **rank · ขอบส้ม**
   ยาว 6 ล้านโทเคนถึงจบโปรเจกต์ · ขอบส้ม = อยู่บนสายวิกฤต


```
{"id": "B07", "size": "L", "skill": "yolo", "difficulty": 3,
"depends_on": ["B02"], "output": "cart.js",
"done": "เพิ่ม/ลด/ลบ/รวมราคา บันทึก localStorage แล้วโหลดคืนหลังรีเฟรช"}
```

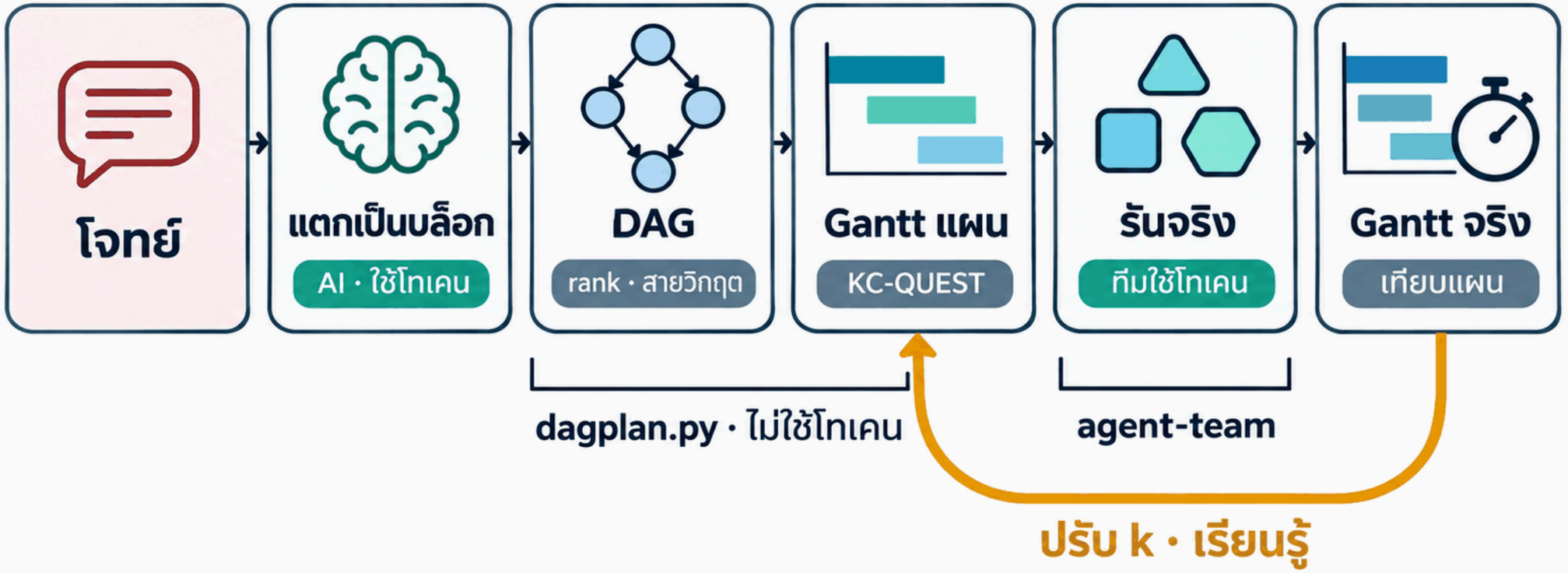

กรอบล่าง = บล็อกเดียวกันใน blocks.json · ไฮไลต์เหลือง = ข้อมูลที่ไม่ได้แสดงในกล่อง (ต้องรอบล็อกไหน · เกณฑ์เสร็จ)

ส่วนที่ 3 จาก 4

# สกิล dag-planner

AI = **สมอง** แตกบล็อก · dagplan.py = **มือ** วาด DAG/Gantt · agent-team = **ทีม** รันจริง

# สกิล dag-planner ทำงานอย่างไร

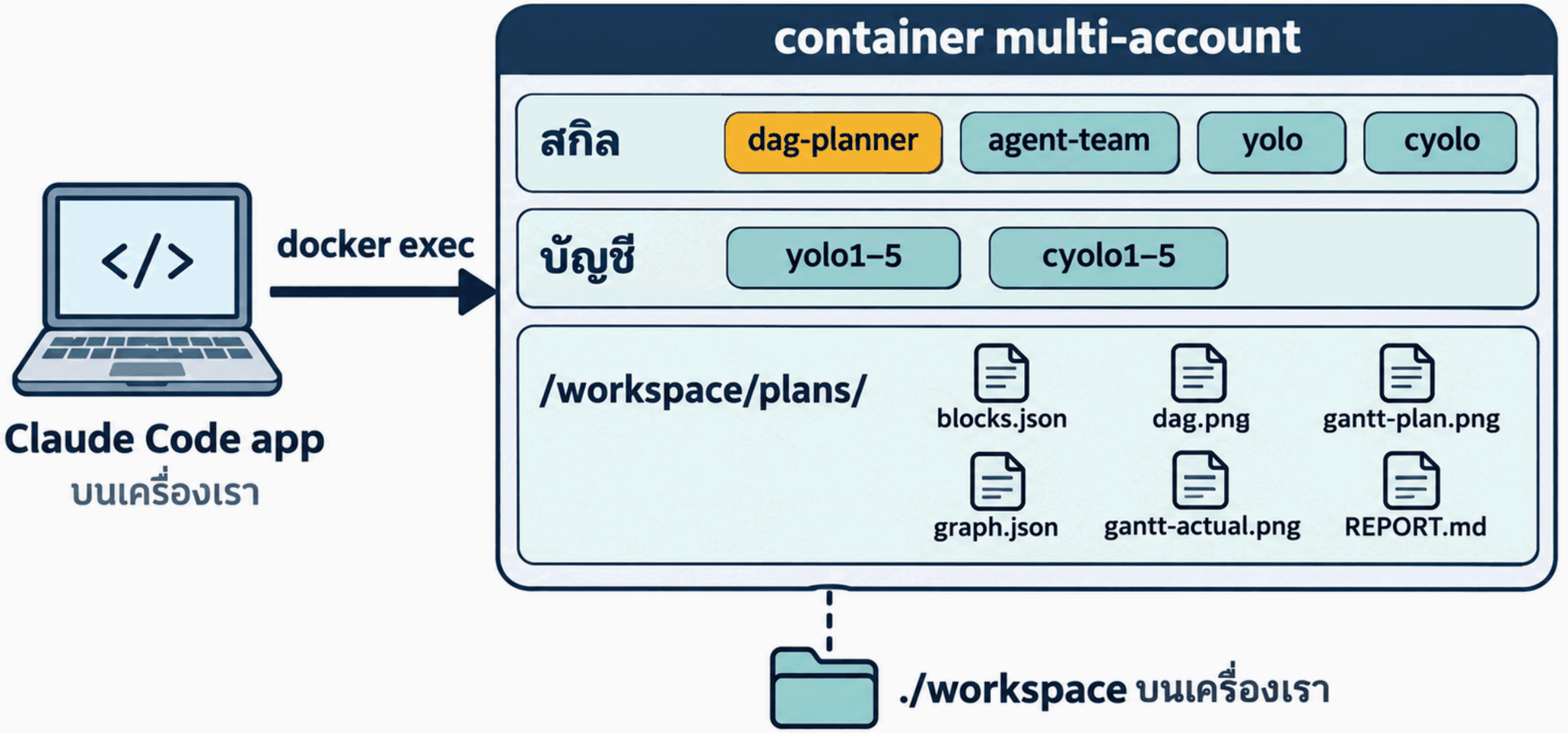


## 3 · ใครทำอะไร

| ใคร | ทำอะไร | โทเคน |
|---|---|---|
| **AI = สมอง** (Claude Code app) | อ่านโจทย์ แตกบล็อก เลือกสกิลและความยาก | ใช้ |
| **dagplan.py = มือ** | ตรวจกราฟ · rank · DAG · KC-QUEST แจกบัญชี · Gantt · export · report | **ไม่ใช้** |
| **agent-team = ทีม** | รันแต่ละบล็อกด้วยสกิลที่ระบุ บนบัญชีจริง | ใช้ |

| คำสั่ง | ผล |
|---|---|
| `doctor` | เครื่องพร้อมไหม |
| `dag` | ตรวจบล็อก + rank + dag.png |
| `plan` | KC-QUEST → gantt-plan.png · PLAN.md |
| `export` · `run` | graph.json → agent-team รันจริง |
| `report` | gantt-actual · compare · REPORT.md · calib.json |

# ใครอยู่ตรงไหน






k

## Gantt แผน = ETA ของแอปนำทาง

report เทียบกับของจริง แล้วคำนวณ k = จริง ÷ แผน แยกตามสกิล เก็บใน calib.json

3 · แผน vs ของจริง

# $k_{skill}$ = เวลาจริง ÷ เวลาแผน

### เวลาต่าง → เรียนรู้ k

report เก็บ k แยกตามสกิลใน calib.json · plan ครั้งถัดไปคูณเวลาด้วย k (KC-QUEST ขั้น ⑥ เรียนรู้)

### บัญชีต่าง → failover ยังทำงาน

export ส่งเป็นฝั่ง yolo/cyolo ไม่ผูกบัญชี agent-team เลือกบัญชีว่างเอง report แสดงส่วนต่าง

ส่วนที่ 4 จาก 4

# ลงมือ 5 ขั้น

สร้างสกิลจากสเปก → ติดตั้ง → วางแผนเหมียวช็อป → รันจริง → what-if

## 4 · ภาพรวมขั้นตอนการทดลอง

| ขั้น | เวลารันจริง |
|---|---|
| **1** สร้างสกิลจากสเปก | ~16.6 นาที |
| **2** ติดตั้งด้วยการคัดลอก | 117 วินาที |
| **3** วางแผนเหมียวช็อป | 82 วินาที |
| **4** รันแผนจริง | 230 วินาที (ทีม 2.8 นาที) |
| **5** what-if ด้วย k ใหม่ | 83 วินาที |

```
05_DAG_Planner/
├── prompt_dag_skill_spec.md  ← สเปก (input 1)
├── tools/                    ← ตัววางแผนเดิม (input 2)
└── _solution/                ← สกิล · TEST-REPORT · แผน meow-shop
```

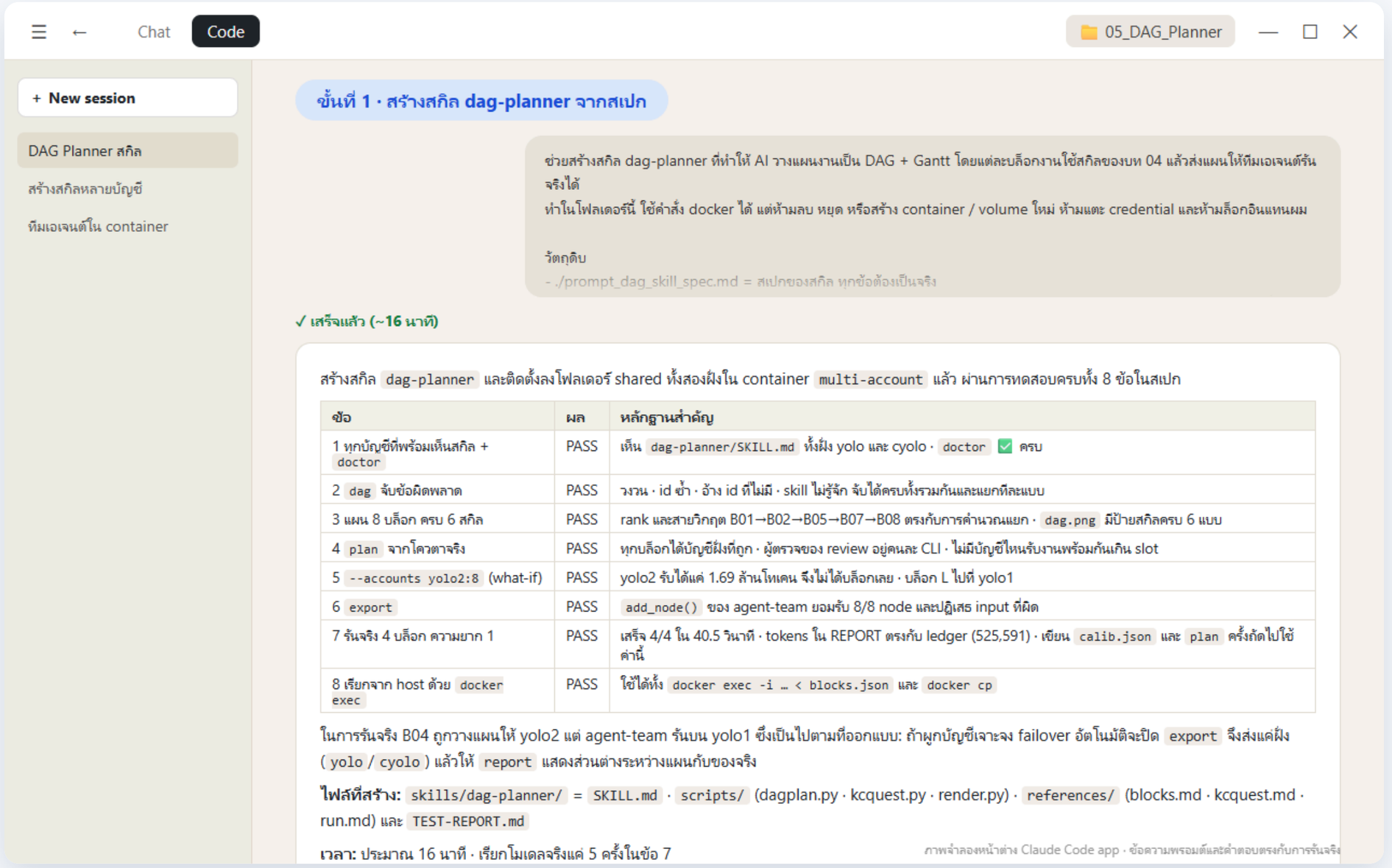
ต้องมี container multi-account + สกิลบท 04 · ล็อกอินอย่างน้อย 2 บัญชี Claude + 2 บัญชี Codex

# 1

# ให้ Claude สร้างสกิลจากสเปก

input: **prompt_dag_skill_spec.md** + **tools/** (ตัววางแผนเดิม) + สกิลบท 04 ใน container (~16.6 นาที)

## ขั้นที่ 1 · 💻 Claude Code app



## ขั้นที่ 1 · พรอมต์ (คัดลอกทั้งกรอบ)

```
ช่วยสร้างสกิล dag-planner ที่ทำให้ AI วางแผนงานเป็น DAG + Gantt โดยแต่ละบล็อกงานใช้สกิลของบท 04 แล้วส่งแผนให้ทีมเอเจนต์รันจริงได้
ทำในโฟลเดอร์นี้ ใช้คำสั่ง docker ได้ แต่ห้ามลบ หยุด หรือสร้าง container / volume ใหม่ ห้ามแตะ credential และห้ามล็อกอินแทนผม

วัตถุดิบ
- ./prompt_dag_skill_spec.md = สเปกของสกิล ทุกข้อต้องเป็นจริง
- ./tools/planner.py และ ./tools/orch.py = ตัววางแผนชุดเดิม (DAG · Gantt · KC-QUEST) ใช้เป็นแหล่งความรู้ ห้ามคัดลอกทั้งไฟล์
- container multi-account ที่กำลังรัน = ที่ทดสอบจริง มีสกิล yolo · cyolo · agent-team ของบท 04 อยู่แล้ว (อ่าน SKILL.md ของสกิลเหล่านั้นใน container ก่อนใช้)

สร้างไว้ใน ./skills/dag-planner (นอก container = ต้นฉบับที่เก็บสำรอง)
ทดสอบ: คัดลอกโฟลเดอร์ไปวางที่ shared ของทั้งสองฝั่งใน container แล้วทดสอบทุกข้อในหัวข้อ "ทดสอบ" ของสเปก
ใช้บัญชีอย่างประหยัด (difficulty 1 กับงานทดสอบ) ข้อที่ต้องใช้บัญชีที่ไม่พร้อม = SKIP
เขียน ./TEST-REPORT.md (นอกโฟลเดอร์ skills) แล้วสรุปเป็นตารางสั้น ๆ: ผลทดสอบ ไฟล์ที่สร้าง และเวลาที่ใช้ แล้วหยุด
```

ขั้นที่ 1 · ผลจริง 8 พ.ย. 2026

# PASS 8 / 8

✻ **Claude** · ขั้นที่ 1 · สร้างสกิล dag-planner จากสเปก · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

สร้างสกิล `dag-planner` และติดตั้งลงโฟลเดอร์ shared ทั้งสองฝั่งใน container `multi-account` แล้ว ผ่านการทดสอบครบทั้ง 8 ข้อในสเปก

| ข้อ | ผล | หลักฐานสำคัญ |
|---|---|---|
| 1 ทุกบัญชีที่พร้อมเห็นสกิล + `doctor` | PASS | เห็น `dag-planner/SKILL.md` ทั้งฝั่ง yolo และ cyolo · `doctor` ✅ ครบ |
| 2 `dag` จับข้อผิดพลาด | PASS | วงวน · id ซ้ำ · อ้าง id ที่ไม่มี · skill ไม่รู้จัก จับได้ครบทั้งรวมกันและแยกทีละแบบ |
| 3 แผน 8 บล็อก ครบ 6 สกิล | PASS | rank และสายวิกฤต B01→B02→B05→B07→B08 ตรงกับการคำนวณแยก · `dag.png` มีป้ายสกิลครบ 6 แบบ |
| 4 `plan` จากโควตาจริง | PASS | ทุกบล็อกได้บัญชีฝั่งที่ถูก · ผู้ตรวจของ review อยู่คนละ CLI · ไม่มีบัญชีไหนรับงานพร้อมกันเกิน slot |
| 5 `--accounts yolo2:8` (what-if) | PASS | yolo2 รับได้แค่ 1.69 ล้านโทเคน จึงไม่ได้บล็อกเลย · บล็อก L ไปที่ yolo1 |
| 6 `export` | PASS | `add_node()` ของ agent-team ยอมรับ 8/8 node และปฏิเสธ input ที่ผิด |
| 7 รันจริง 4 บล็อก ความยาก 1 | PASS | เสร็จ 4/4 ใน 40.5 วินาที · tokens ใน REPORT ตรงกับ ledger (525,591) · เขียน `calib.json` และ `plan` ครั้งถัดไปใช้ค่านี้ |
| 8 เรียกจาก host ด้วย `docker exec` | PASS | ใช้ได้ทั้ง `docker exec -i … < blocks.json` และ `docker cp` |

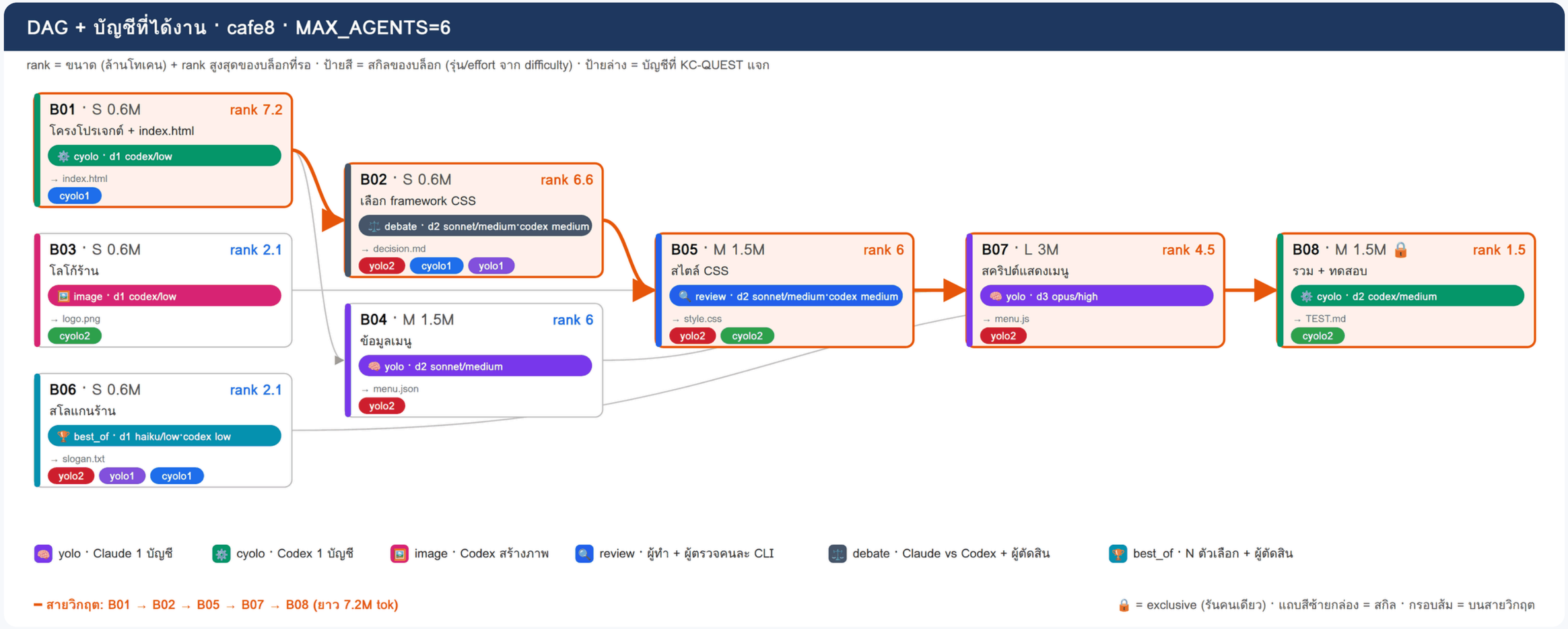
ในการรันจริง B04 ถูกวางแผนให้ yolo2 แต่ agent-team รันบน yolo1 ซึ่งเป็นไปตามที่ออกแบบ: ถ้าผูกบัญชีเจาะจง failover อัตโนมัติจะปิด `export` จึงส่งแค่ฝั่ง ( `yolo` / `cyolo` ) แล้วให้ `report` แสดงส่วนต่างระหว่างแผนกับของจริง

**ไฟล์ที่สร้าง:** `skills/dag-planner/` = `SKILL.md` · `scripts/` (dagplan.py · kcquest.py · render.py) · `references/` (blocks.md · kcquest.md · run.md) และ `TEST-REPORT.md`

**เวลา:** ประมาณ 16 นาที · เรียกโมเดลจริงแค่ 5 ครั้งในข้อ 7

คำตอบจริงเป็นภาษาอังกฤษ แปลและจัดหน้าใหม่ · รายงานเต็ม _solution/TEST-REPORT.md

# ขั้นที่ 1 · ภาพจริงจากการทดสอบ: แผน 8 บล็อกครบ 6 สกิล




ทุกกล่องมีป้ายสีของสกิล + รุ่น/effort · ป้ายล่าง = บัญชีที่ KC-QUEST แจก · บล็อกหลายบัญชีมีหลายป้าย

## ขั้นที่ 1 · ไฟล์ที่ Claude สร้าง

```
skills/dag-planner/
├── SKILL.md            ← ตาราง "ต้องการ → คำสั่ง" · ขั้นตอน · กติกาแตกบล็อก · ตารางเลือกสกิล · เรียกจาก host
├── scripts/            ← dagplan.py (คำสั่งหลัก) · kcquest.py (KC-QUEST ที่ดึงจาก orch.py) · render.py (วาด DAG · Gantt · compare)
└── references/         ← blocks.md (ฟิลด์ + ตัวอย่าง 8 บล็อก) · kcquest.md (สูตรย่อ) · run.md (รันจริงและรายงาน)
```

เฉลยอยู่ที่ _solution/skills/dag-planner/

# 2

# ติดตั้งสกิลด้วยการคัดลอกโฟลเดอร์

คัดลอก **dag-planner** ลง shared/skills ทั้งสองฝั่ง · ไม่ต้องรันสคริปต์ติดตั้ง (117 วินาที)

## ขั้นที่ 2 · 💻 Claude Code app

```
ติดตั้งสกิลจาก ./skills/dag-planner ลง container multi-account ด้วยการคัดลอกโฟลเดอร์อย่างเดียว
- คัดลอกโฟลเดอร์ dag-planner ไปวางที่ /root/ai-accounts/.Claude/shared/skills และ /root/ai-accounts/.Codex/shared/skills ใน container
- เปิดแต่ละบัญชีหนึ่งครั้งด้วย yoloN --version และ cyoloN --version (wrapper จะเชื่อมสกิลให้แต่ละบัญชีเอง)
- ตรวจว่าทุกบัญชี yolo1-5 และ cyolo1-5 มองเห็น dag-planner/SKILL.md แล้วรันคำสั่ง doctor ของสกิล
สรุปเป็นตาราง ฝั่ง | มองเห็นสกิล · ผล doctor แล้วหยุด
```



## ขั้นที่ 2 · ผลจริง

✻ **Claude** · ขั้นที่ 2 · ติดตั้งสกิลด้วยการคัดลอกโฟลเดอร์ · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

ติดตั้ง dag-planner ใน `multi-account` เรียบร้อยแล้ว ทุกบัญชีมองเห็น `dag-planner/SKILL.md` และ doctor ผ่านทั้งสองฝั่ง

| ฝั่ง | มองเห็นสกิล · ผล doctor |
|---|---|
| Claude ( `.Claude/shared/skills` ) | yolo1–5 เห็นครบ 5/5 · doctor ผ่าน "พร้อมใช้ ✅ " (exit 0) |
| Codex ( `.Codex/shared/skills` ) | cyolo1–5 เห็นครบ 5/5 · doctor ผ่าน "พร้อมใช้ ✅ " (exit 0) |

- **สิ่งที่ทำ:** คัดลอกด้วย `docker cp` ไปทั้งสองที่ แล้วเปิด `yoloN --version` (Claude Code 2.1.319) และ `cyoloN --version` (codex-cli 0.167.0) ครบทุกบัญชี
- **ของเดิม:** ใน container มี dag-planner อยู่ก่อนแล้ว (จากขั้นที่ 1) ตรงกับไฟล์ local ทุกไฟล์ (ตรวจ md5) จึงคัดลอกทับได้โดยไม่มีอะไรเปลี่ยน

> yolo1–5 และ cyolo1–5 เห็น dag-planner/SKILL.md ครบ · doctor ผ่านทั้งสองฝั่ง

ใช้สกิลที่เตรียมไว้: เปลี่ยน ./skills/dag-planner เป็น ./_solution/skills/dag-planner · หรือลากวางใน JupyterLab แบบบท 04

# 3

# วางแผนโจทย์ "เหมียวช็อป"

Claude Code app **เจอสกิล dag-planner** เอง → บล็อกงาน + DAG + Gantt แผน (82 วินาที)




## เว็บร้านอาหารแมวหน้าเดียว

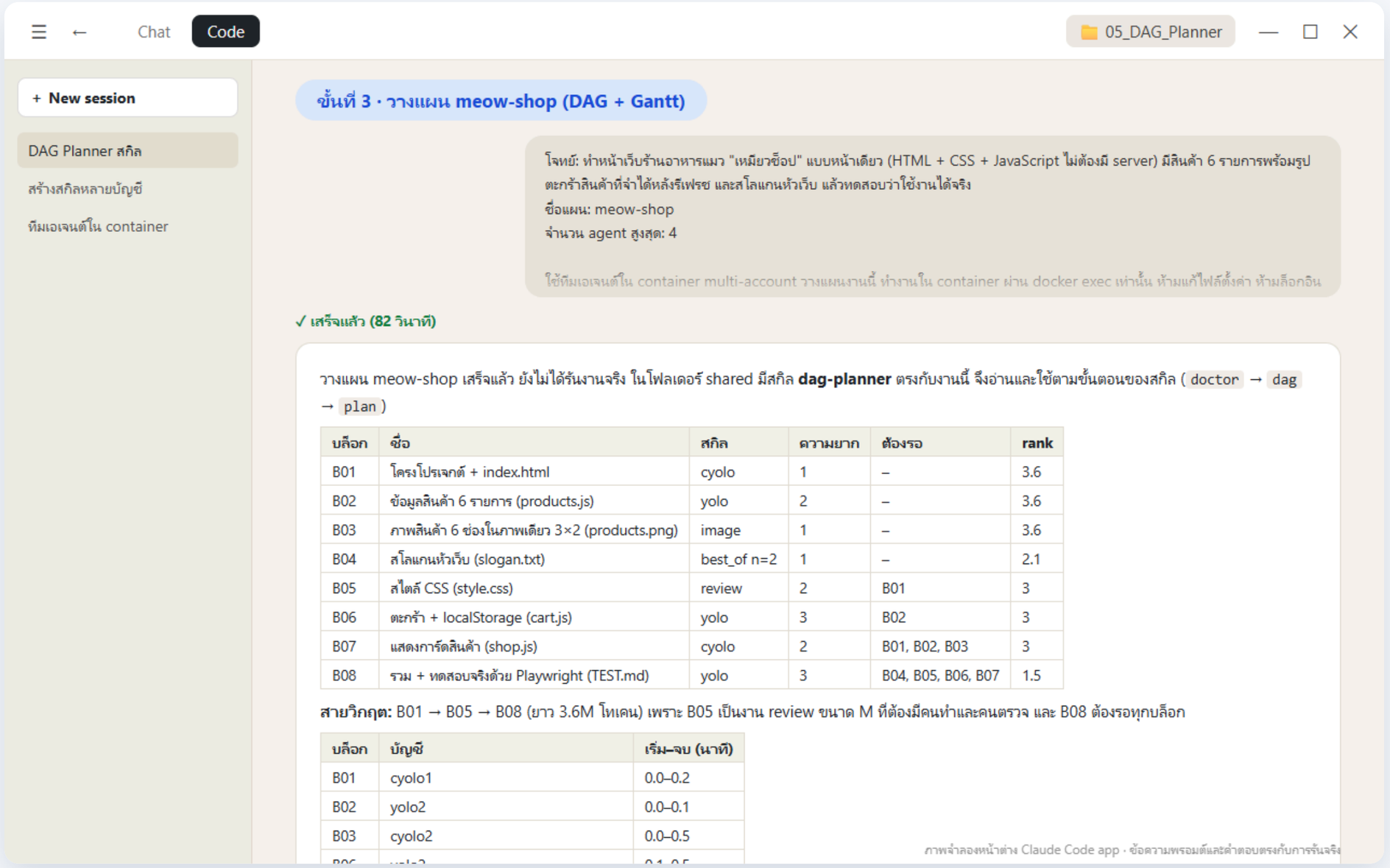
สินค้า 6 รายการพร้อมรูป · ตะกร้าจำได้หลังรีเฟรช · สโลแกนหัวเว็บ · ทดสอบใช้งานได้จริง

## ขั้นที่ 3 · พรอมต์

```
โจทย์: ทำหน้าเว็บร้านอาหารแมว "เหมียวช็อป" แบบหน้าเดียว (HTML + CSS + JavaScript ไม่ต้องมี server) มีสินค้า 6 รายการพร้อมรูป ตะกร้าสินค้าที่จำได้หลังรีเฟรช และสโลแกนหัวเว็บ แล้วทดสอบว่าใช้งานได้จริง
ชื่อแผน: meow-shop
จำนวน agent สูงสุด: 4

ใช้ทีมเอเจนต์ใน container multi-account วางแผนงานนี้ ทำงานใน container ผ่าน docker exec เท่านั้น ห้ามแก้ไฟล์ตั้งค่า ห้ามล็อกอินแทนฉัน
ก่อนเริ่ม ดูว่าใน container มีสกิลที่ช่วยงานนี้ได้หรือไม่ (/root/ai-accounts/.Claude/shared/skills) ถ้ามีให้อ่านและใช้
1. แตกโจทย์เป็นบล็อกงาน 6-10 บล็อก แต่ละบล็อกบอกสกิลที่ใช้ทำ ความยาก ไฟล์ผล และเกณฑ์เสร็จ
2. วาด DAG แล้ววางแผนว่าบล็อกไหนให้บัญชีไหน เมื่อไร (Gantt) จากโควตาจริง ยังไม่ต้องรันงานจริง
ตอบภาษาไทยสั้น ๆ: ตาราง บล็อก | ชื่อ | สกิล | ความยาก | ต้องรอ | rank · สายวิกฤตพร้อมเหตุผล 1 บรรทัด · ตาราง บล็อก | บัญชี | เริ่ม–จบ (นาที) · path ของ dag.png และ gantt-plan.png
```

## ขั้นที่ 3 · 💻 Claude Code app



# ขั้นที่ 3 · ผลจริง · 8 บล็อก 5 สกิล · แผน 1.7 นาที

✻ **Claude** · ขั้นที่ 3 · วางแผน meow-shop (DAG + Gantt) · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

วางแผน meow-shop เสร็จแล้ว ยังไม่ได้รันงานจริง ในโฟลเดอร์ shared มีสกิล **dag-planner** ตรงกับงานนี้ จึงอ่านและใช้ตามขั้นตอนของสกิล ( `doctor` → `dag` → `plan` )

| บล็อก | ชื่อ | สกิล | ความยาก | ต้องรอ | rank |
|---|---|---|---|---|---|
| B01 | โครงโปรเจกต์ + index.html | cyolo | 1 | – | 3.6 |
| B02 | ข้อมูลสินค้า 6 รายการ (products.js) | yolo | 2 | – | 3.6 |
| B03 | ภาพสินค้า 6 ช่องในภาพเดียว 3×2 (products.png) | image | 1 | – | 3.6 |
| B04 | สโลแกนหัวเว็บ (slogan.txt) | best_of n=2 | 1 | – | 2.1 |
| B05 | สไตล์ CSS (style.css) | review | 2 | B01 | 3 |
| B06 | ตะกร้า + localStorage (cart.js) | yolo | 3 | B02 | 3 |
| B07 | แสดงการ์ดสินค้า (shop.js) | cyolo | 2 | B01, B02, B03 | 3 |
| B08 | รวม + ทดสอบจริงด้วย Playwright (TEST.md) | yolo | 3 | B04, B05, B06, B07 | 1.5 |

**สายวิกฤต:** B01 → B05 → B08 (ยาว 3.6M โทเคน) เพราะ B05 เป็นงาน review ขนาด M ที่ต้องมีคนทำและคนตรวจ และ B08 ต้องรอทุกบล็อก

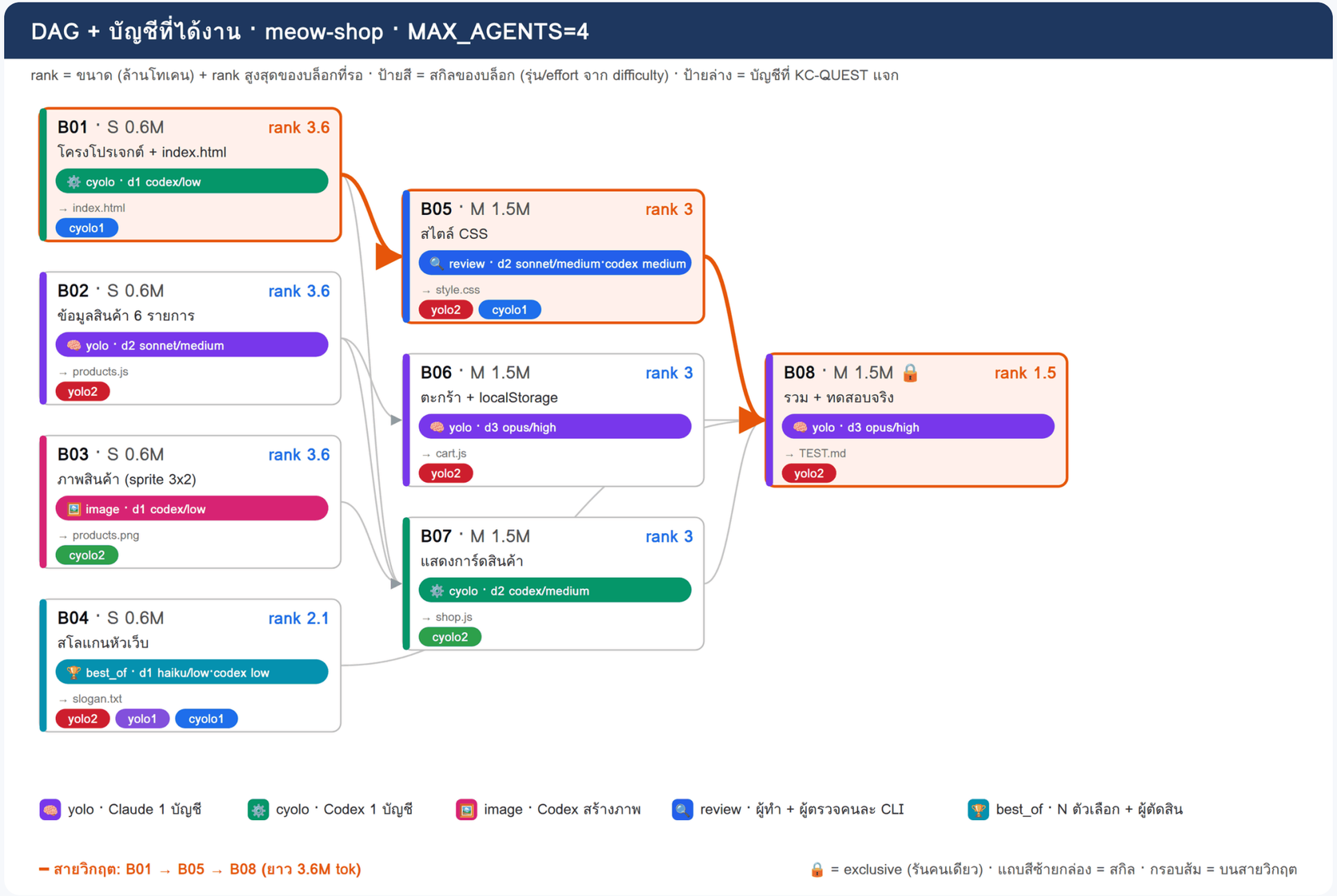
| บล็อก | บัญชี | เริ่ม–จบ (นาที) |
|---|---|---|
| B01 | cyolo1 | 0.0–0.2 |
| B02 | yolo2 | 0.0–0.1 |
| B03 | cyolo2 | 0.0–0.5 |
| B06 | yolo2 | 0.1–0.5 |
| B05 | yolo2 (ทำ) + cyolo1 (ตรวจ) | 0.2–1.0 |
| B07 | cyolo2 | 0.5–1.1 |
| B04 | yolo2 + yolo1 (ตอบ) + cyolo1 (ตัดสิน) | 1.0–1.3 |
| B08 | yolo2 | 1.3–1.7 |

- agent พร้อมกันสูงสุด 4 ตัว ใช้เวลารวมประมาณ **1.7 นาที**
- ระบบเลือก yolo2 บ่อยที่สุด เพราะโควตาจะรีเซ็ตเร็วที่สุด (26.6 ชม.) ใช้ให้คุ้มก่อนรีเซ็ต

**ไฟล์ผล:** `./workspace/plans/meow-shop/dag.png` · `gantt-plan.png` · `PLAN.md` (เหตุผลที่เลือกแต่ละบัญชี)

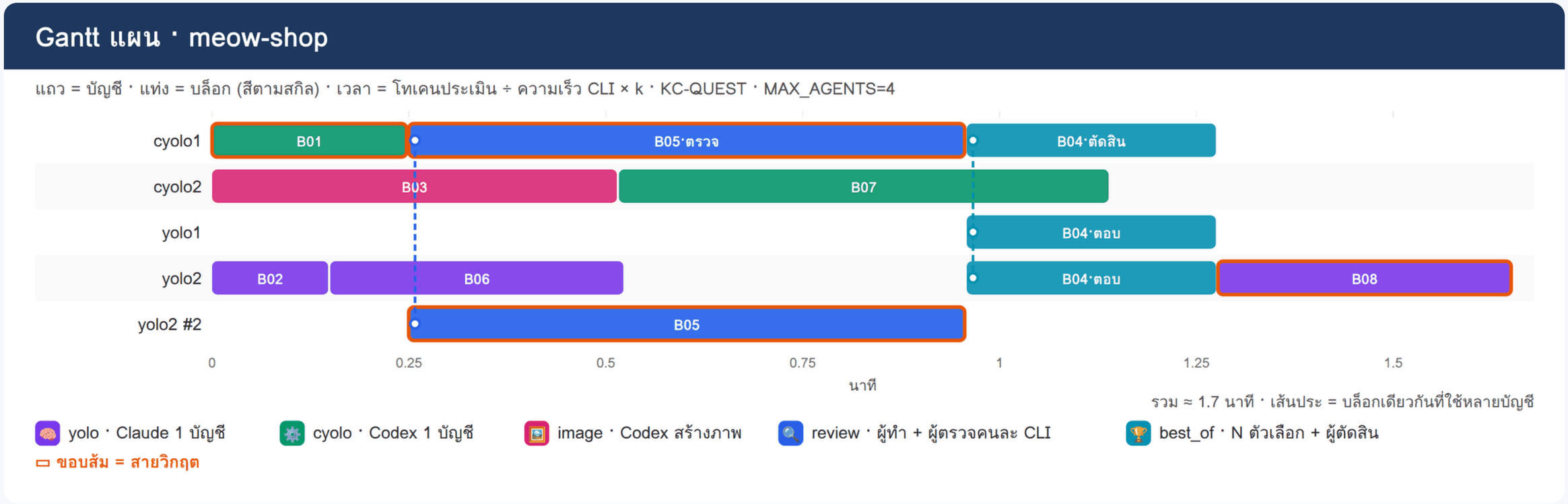
ถ้าจะรันจริงต่อ ใช้คำสั่ง `export` , `run` , `report` ตามลำดับ ซึ่งจะใช้โทเคนจริง ให้บอกก่อนแล้วจะเริ่มรัน

## ขั้นที่ 3 · dag.png (ภาพจริง)




สายวิกฤต B01 → B05 → B08 = 0.6 + 1.5 + 1.5 = 3.6 ล้านโทเคน · B05 review: yolo2 ทำ + cyolo1 ตรวจ · B04 best_of: 3 บัญชี

## ขั้นที่ 3 · gantt-plan.png (ภาพจริง)




บล็อกหลายบัญชีมีแท่งหลายแถวเชื่อมด้วยเส้นประ · yolo2 ได้งานมากสุดเพราะโควตาจะรีเซ็ตเร็วสุด (26.6 ชม.)

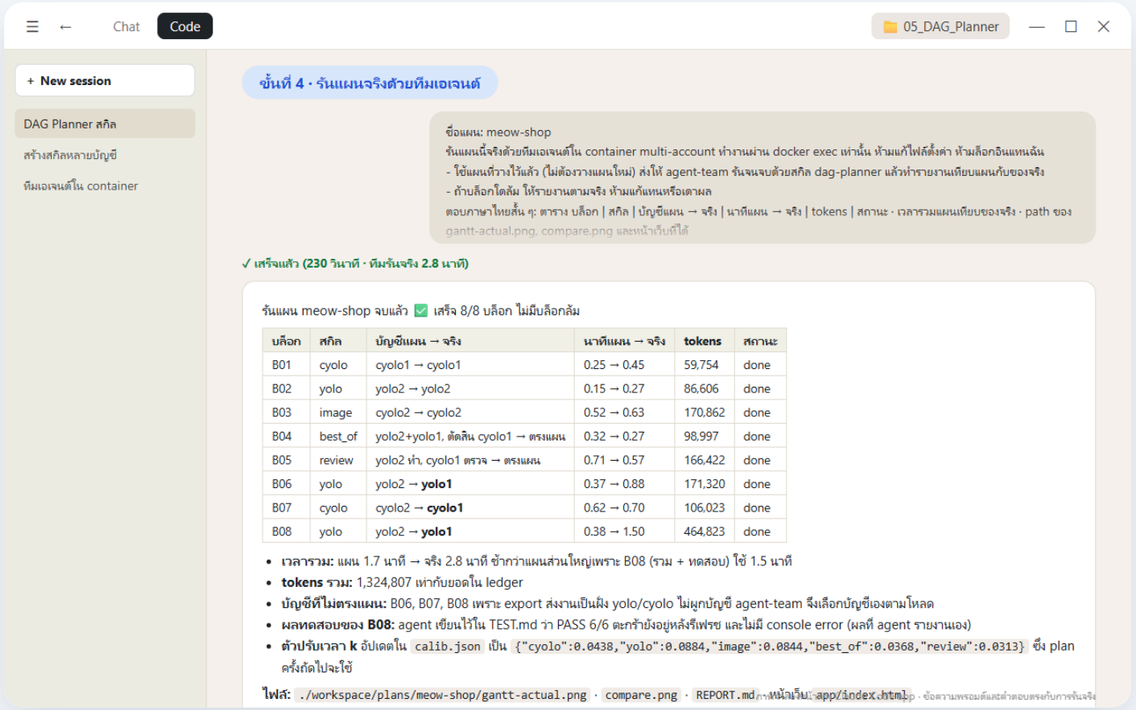
# 4

# รันแผนจริงด้วยทีมเอเจนต์

export → run → report · ทีมรันจริง **2.8 นาที** · เสร็จ 8/8 บล็อก

## ขั้นที่ 4 · 💻 Claude Code app

```
ชื่อแผน: meow-shop
รันแผนนี้จริงด้วยทีมเอเจนต์ใน container multi-account ทำงานผ่าน docker exec เท่านั้น ห้ามแก้ไฟล์ตั้งค่า ห้ามล็อกอินแทนฉัน
- ใช้แผนที่วางไว้แล้ว (ไม่ต้องวางแผนใหม่) ส่งให้ agent-team รันจนจบด้วยสกิล dag-planner แล้วทำรายงานเทียบแผนกับของจริง
- ถ้าบล็อกใดล้ม ให้รายงานตามจริง ห้ามแก้แทนหรือเดาผล
ตอบภาษาไทยสั้น ๆ: ตาราง บล็อก | สกิล | บัญชีแผน → จริง | นาทีแผน → จริง | tokens | สถานะ · เวลารวมแผนเทียบของจริง · path ของ gantt-actual.png, compare.png และหน้าเว็บที่ได้
```




> "ใช้แผนที่วางไว้แล้ว" สำคัญ ไม่เช่นนั้น Claude อาจวางแผนใหม่ทับแผนขั้นที่ 3

## ขั้นที่ 4 · ผลจริง

✻ **Claude** · ขั้นที่ 4 · รันแผนจริงด้วยทีมเอเจนต์ · คำตอบจริง (จัดหน้าใหม่ให้อ่านง่าย)

รันแผน meow-shop จบแล้ว ✅ เสร็จ 8/8 บล็อก ไม่มีบล็อกล้ม

| บล็อก | สกิล | บัญชีแผน → จริง | นาทีแผน → จริง | tokens | สถานะ |
|---|---|---|---|---|---|
| B01 | cyolo | cyolo1 → cyolo1 | 0.25 → 0.45 | 59,754 | done |
| B02 | yolo | yolo2 → yolo2 | 0.15 → 0.27 | 86,606 | done |
| B03 | image | cyolo2 → cyolo2 | 0.52 → 0.63 | 170,862 | done |
| B04 | best_of | yolo2+yolo1, ตัดสิน cyolo1 → ตรงแผน | 0.32 → 0.27 | 98,997 | done |
| B05 | review | yolo2 ทำ, cyolo1 ตรวจ → ตรงแผน | 0.71 → 0.57 | 166,422 | done |
| B06 | yolo | yolo2 → **yolo1** | 0.37 → 0.88 | 171,320 | done |
| B07 | cyolo | cyolo2 → **cyolo1** | 0.62 → 0.70 | 106,023 | done |
| B08 | yolo | yolo2 → **yolo1** | 0.38 → 1.50 | 464,823 | done |

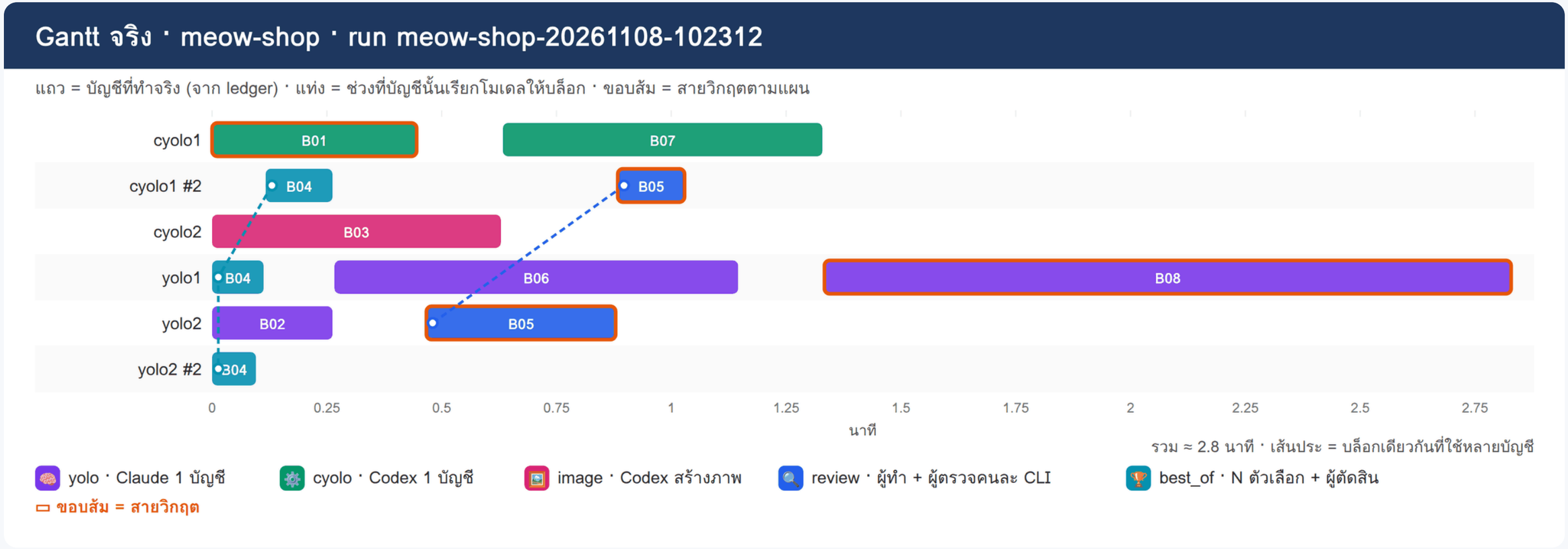
- **เวลารวม:** แผน 1.7 นาที → จริง 2.8 นาที ช้ากว่าแผนส่วนใหญ่เพราะ B08 (รวม + ทดสอบ) ใช้ 1.5 นาที
- **tokens รวม:** 1,324,807 เท่ากับยอดใน ledger
- **บัญชีที่ไม่ตรงแผน:** B06, B07, B08 เพราะ export ส่งงานเป็นฝั่ง yolo/cyolo ไม่ผูกบัญชี agent-team จึงเลือกบัญชีเองตามโหลด
- **ผลทดสอบของ B08:** agent เขียนไว้ใน TEST.md ว่า PASS 6/6 ตะกร้ายังอยู่หลังรีเฟรช และไม่มี console error (ผลที่ agent รายงานเอง)
- **ตัวปรับเวลา k** อัปเดตใน `calib.json` เป็น `{"cyolo":0.0438,"yolo":0.0884,"image":0.0844,"best_of":0.0368,"review":0.0313}` ซึ่ง plan ครั้งถัดไปจะใช้

**ไฟล์:** `./workspace/plans/meow-shop/gantt-actual.png` · `compare.png` · `REPORT.md` · หน้าเว็บ `app/index.html`

## ขั้นที่ 4 · แผน vs ของจริงตามเวลา (จาก plan.json + ledger)

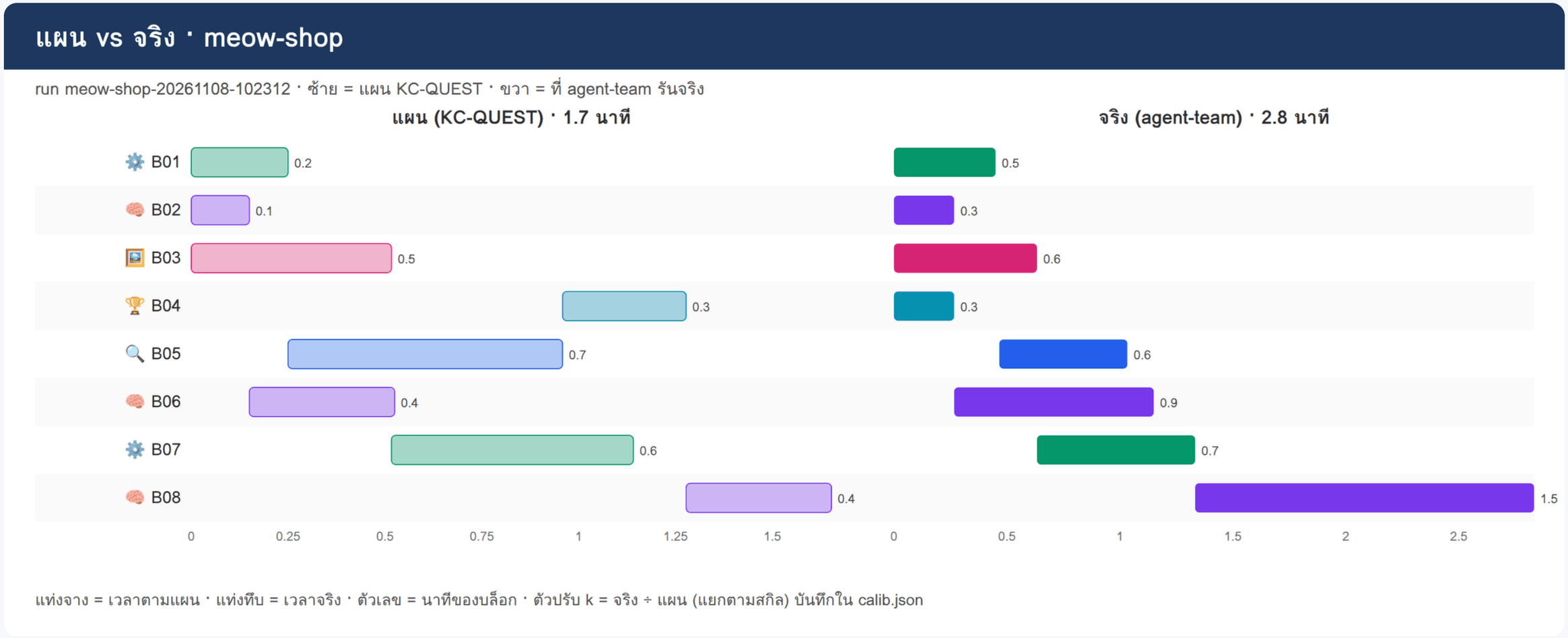


## ขั้นที่ 4 · gantt-actual.png (ภาพจริง)



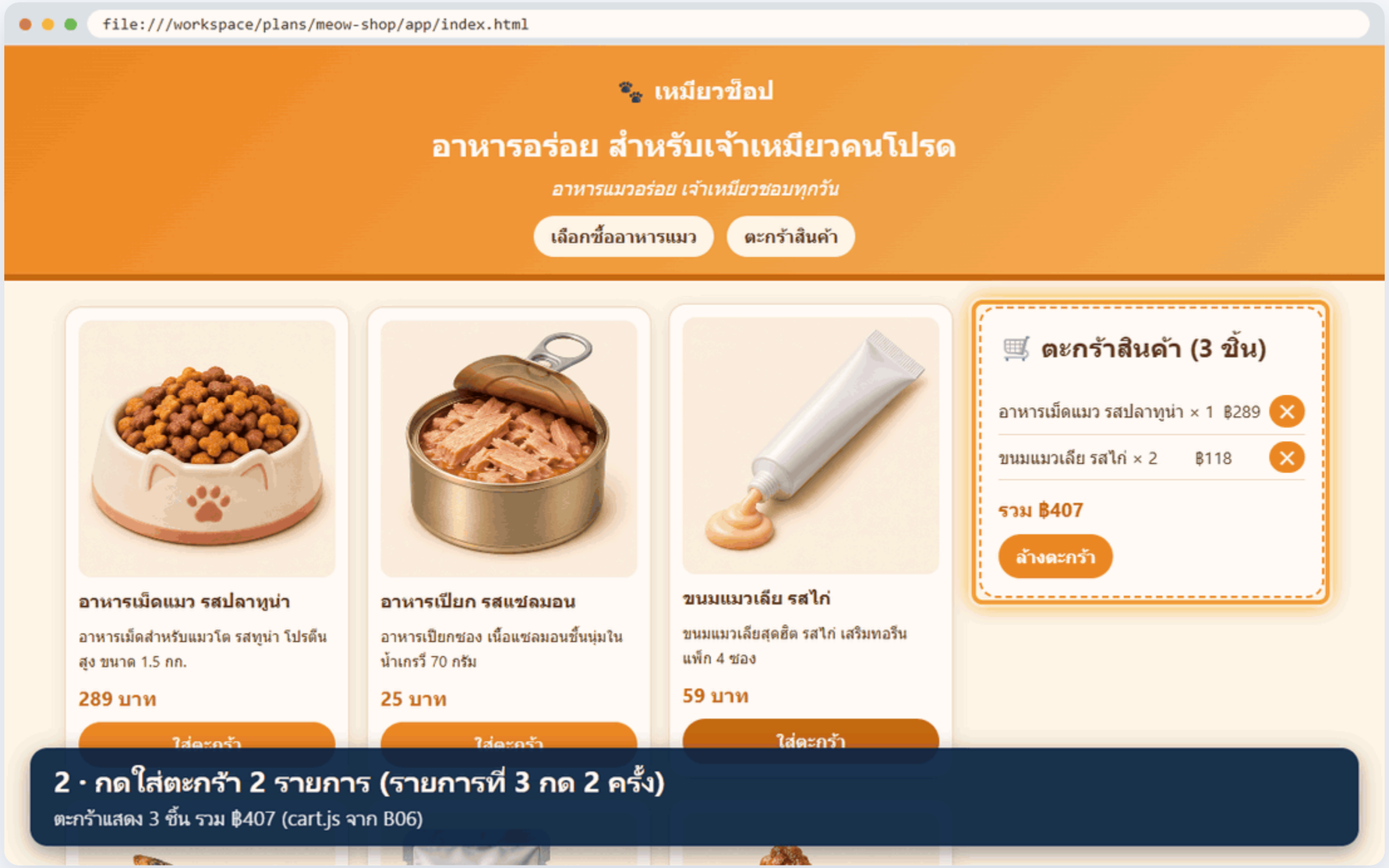

review ของจริงเป็นลำดับ (ทำ แล้วตรวจ) · B06 B07 B08 ไปบัญชีอื่นเพราะ export ส่งเป็นฝั่ง · B08 ลากยาวถึงนาทีที่ 2.8

## ขั้นที่ 4 · compare.png (ภาพจริง)

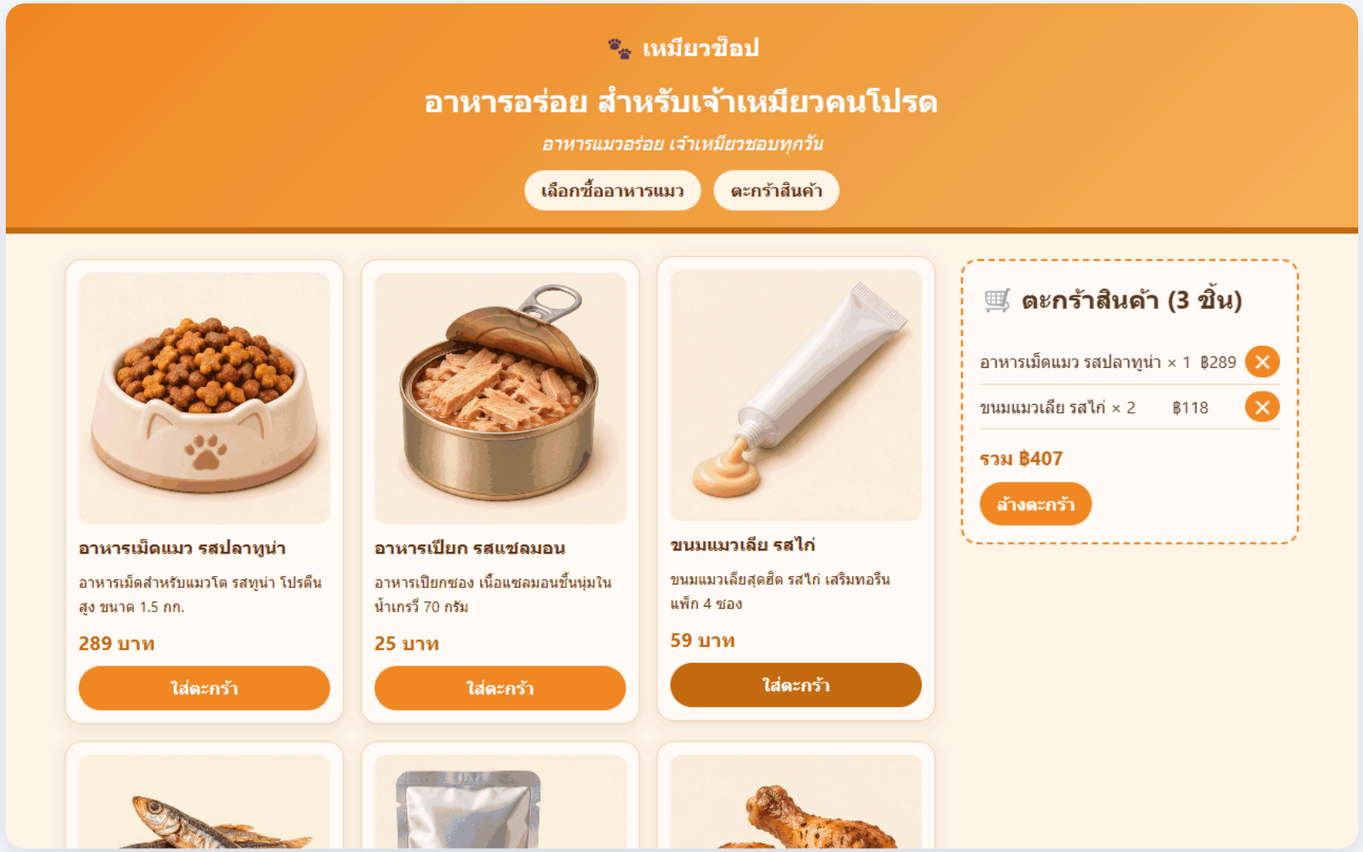



B08 แผน 0.4 → จริง 1.5 นาที · k ใหม่: yolo 0.0884 · cyolo 0.0438 · image 0.0844 · best_of 0.0368 · review 0.0313

## ขั้นที่ 4 · หน้าเว็บที่ทีมสร้าง · ทดสอบจริงด้วย Playwright



ขั้นที่ 4 · บทเรียนจากของจริง




## ลูกศรที่ขาด = งานที่ไม่ตรงกัน

B08 บันทึกว่า **รูปบางช่องไม่ตรงชื่อสินค้า** เช่น "อาหารเปียก รสแซลมอน" ได้รูปกระป๋องทูน่า

สาเหตุ: B03 (ภาพ) **ไม่ได้รอ** B02 (ข้อมูลสินค้า) คนวาดภาพจึงไม่รู้ว่าแต่ละช่องคืออะไร · ตัดลูกศรได้เฉพาะงานที่ไม่ใช้ข้อมูลของกันจริง

# 5

# what-if ด้วย k ที่เรียนรู้แล้ว

plan ใหม่ 3 แบบ **ไม่รันทีมจริง** ไม่ใช้โทเคนของทีม (83 วินาที)

## ขั้นที่ 5 · 💻 Claude Code app

```
ชื่อแผน: meow-shop
วางแผนใหม่ 2 แบบ (ยังไม่รันจริง) โดยใช้ตัวปรับ k ที่ได้จากการรันจริงรอบที่แล้ว แล้วเทียบกับแผนเดิม
ก. จำนวน agent สูงสุด 2
ข. yolo1 เหลือโควตาแค่ 5%
ทำผ่าน docker exec ใน container multi-account ด้วยสกิล dag-planner
ตอบเป็นตาราง แบบ | เวลารวม (นาที) | บล็อกบนสายวิกฤตไปบัญชีไหน | อะไรเปลี่ยน พร้อม path ของ gantt ทั้ง 2 แบบ
```

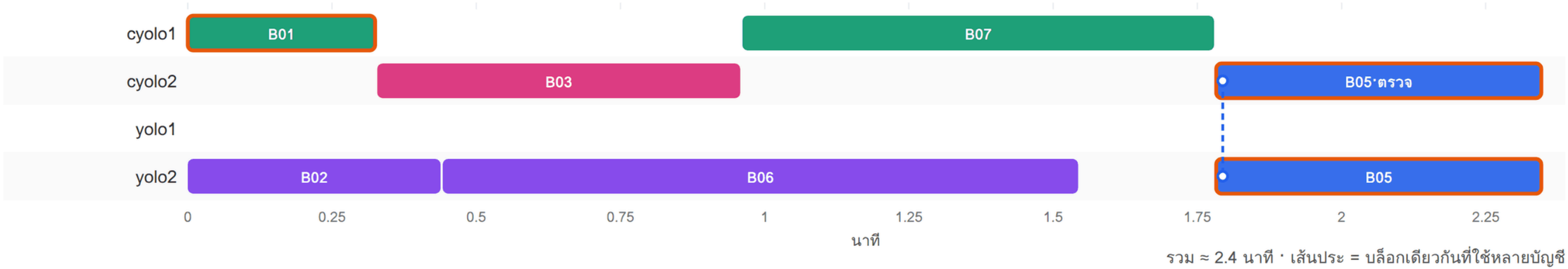


## ขั้นที่ 5 · ผลจริง

| แบบ | เวลารวม | สิ่งที่เห็น |
|---|---|---|
| แผนเดิม (k จากขั้นที่ 1) | 1.7 นาที | ประเมินต่ำไป |
| **k ใหม่ · agent 4** | **2.8 นาที** | **เท่ากับเวลาจริง** |
| ก. agent สูงสุด 2 | 2.4 นาที · เสร็จแค่ 6/8 | B04 best_of ต้องใช้ 3 บัญชี → ไม่มีใครรับ · B08 ติดตาม |
| ข. yolo1 เหลือ 5% | 2.9 นาที | yolo1 ไม่ได้บล็อกเลย สายวิกฤตไม่กระทบ |

### Gantt แผน · meow-shop

แถว = บัญชี · แท่ง = บล็อก (สีตามสกิล) · เวลา = โทเคนประเมิน ÷ ความเร็ว CLI × k · KC-QUEST · MAX_AGENTS=2




รวม ≈ 2.4 นาที · เส้นประ = บล็อกเดียวกันที่ใช้หลายบัญชี

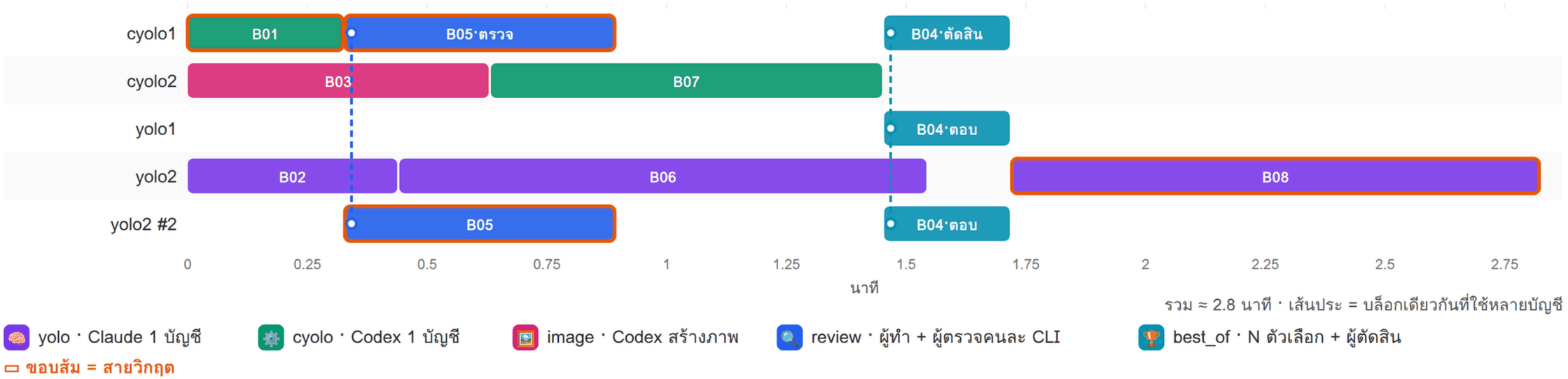
yolo · Claude 1 บัญชี · cyolo · Codex 1 บัญชี · image · Codex สร้างภาพ · review · ผู้ทำ + ผู้ตรวจคนละ CLI · best_of · N ตัวเลือก + ผู้ตัดสิน

▭ ขอบส้ม = สายวิกฤต

## ขั้นที่ 5 · แผนหลังเรียนรู้ k = 2.8 นาที (ของจริง 2.8 นาที)

### Gantt แผน · meow-shop

แถว = บัญชี · แท่ง = บล็อก (สีตามสกิล) · เวลา = โทเคนประเมิน ÷ ความเร็ว CLI × k · KC-QUEST · MAX_AGENTS=4




รวม ≈ 2.8 นาที · เส้นประ = บล็อกเดียวกันที่ใช้หลายบัญชี

yolo · Claude 1 บัญชี | cyolo · Codex 1 บัญชี | image · Codex สร้างภาพ | review · ผู้ทำ + ผู้ตรวจคนละ CLI | best_of · N ตัวเลือก + ผู้ตัดสิน

▭ ขอบส้ม = สายวิกฤต

สรุป

# ไม่มีแผน vs มี dag-planner

| | บท 04 | บท 05 |
|---|---|---|
| **ใครเขียน graph.json** | คน / AI เขียนเอง | export จาก blocks.json |
| **แต่ละงานใช้สกิลอะไร** | ระบุเองทีละ node | ทุกบล็อกมี skill + difficulty |
| **ใครทำบล็อกไหน เมื่อไร** | ไม่รู้จนกว่าจะรัน | Gantt แผน (KC-QUEST + โควตาจริง) |
| **เสร็จเมื่อไร** | ไม่รู้ | 1.7 → จริง 2.8 → แผนใหม่ 2.8 นาที |
| **what-if** | ต้องรันจริง | plan ใหม่ ไม่ใช้โทเคนของทีม |

ประเด็นสำคัญ

# จำ 3 ข้อนี้

1 · บล็อก

ทุกบล็อกบอก **สกิล** ความยาก ไฟล์ผล และเกณฑ์เสร็จ

2 · แผน

**DAG + Gantt** บอกว่าใครทำอะไร เมื่อไร ก่อนเสียโทเคน

3 · เรียนรู้

รันจริงแล้ว **k** ทำให้แผนครั้งถัดไปแม่นขึ้น

ข้อควรรู้

# ข้อควรรู้

### แผนเป็นค่าประมาณ

แผนแรกต่างจากจริงได้มาก calib.json ช่วยให้แม่นขึ้น

### รันจริงใช้โทเคน

สกิลให้ถามผู้ใช้ก่อน run และเริ่มจากความยากต่ำ

### บัญชีอาจไม่ตรงแผน

export ส่งเป็นฝั่ง ถ้าจะห้ามบัญชีไหนให้ตัดจาก --accounts

### ลูกศรที่ขาด

ตัด dependency เฉพาะงานที่ไม่ใช้ข้อมูลของกันจริง

### บล็อกหลายบัญชี

best_of n=2 ใช้ 3 บัญชีพร้อมกัน agent สูงสุดต้องไม่ต่ำกว่านั้น

### บอกให้ใช้แผนเดิม

ก่อนรันจริง ไม่เช่นนั้น Claude อาจวางแผนใหม่ทับ

คำสั่งที่ใช้บ่อย

## ใน container

```
D=/root/ai-accounts/.Claude/shared/skills/dag-planner/scripts/dagplan.py
python3 $D doctor                                  # เครื่องพร้อมไหม
python3 $D dag meow-shop --blocks blocks.json      # ตรวจบล็อก + rank + dag.png
python3 $D plan meow-shop --max-agents 4           # Gantt แผนจากโควตาจริง
python3 $D plan meow-shop --accounts yolo1:5,yolo2,cyolo1,cyolo2   # what-if (สมมติโควตา)
python3 $D export meow-shop && python3 $D run meow-shop --detach   # รันจริง (ใช้โทเคน)
python3 $D report meow-shop                        # Gantt จริง + แผน vs จริง + calib.json
```

## จาก host

```
docker exec multi-account cat /root/ai-accounts/.Claude/shared/skills/dag-planner/SKILL.md
docker exec -i multi-account python3 /root/ai-accounts/.Claude/shared/skills/dag-planner/scripts/dagplan.py dag meow-shop --blocks - < blocks.json
```

เช็กลิสต์

# ทำครบหรือยัง?

- [ ] มีโฟลเดอร์ `skills/dag-planner` และ TEST-REPORT ผ่าน (ขั้นที่ 1)
- [ ] ทุกบัญชีเห็นสกิลและ doctor ผ่าน (ขั้นที่ 2)
- [ ] ได้ dag.png และ gantt-plan.png ที่มีป้ายสกิลทุกบล็อก (ขั้นที่ 3)
- [ ] ทีมรันจริงจนจบ ได้ gantt-actual · compare · หน้าเว็บ (ขั้นที่ 4)
- [ ] แผนหลังเรียนรู้ k ใกล้เวลาจริง และลอง what-if แล้ว (ขั้นที่ 5)

สรุป

# วางแผนก่อน แล้วให้ทีมทำตามแผน

บล็อกบอก **สกิล** · DAG บอก **ลำดับ** · Gantt บอก **ใคร เมื่อไร** · ของจริงสอน **k**

ต่อด้วยบท 06 · KC-QUEST เพื่อดูสมการแจกบล็อกให้บัญชีแบบละเอียด

รายละเอียดทุกขั้นตอนและพรอมต์เต็มอยู่ใน README.md

